

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK 4



DISUSUN OLEH :
FATHUR RAMADHAN FAHMI
2311533012

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2024

Membuat Fungsi CRUD untuk OrderDetailFrame

1. Membuat Class OrderDetailDAO

```
1 package DAO;
2
3 import java.util.List;
4
5
6 public interface OrderDetailDAO {
7     void save(OrderDetail orderDetail);
8     List<OrderDetail> findAll();
9     void update(OrderDetail orderDetail);
10    void delete(String idOrderDetail);
11    List<OrderDetail> show();
12 }
```

2. Membuat Class OrderDetailRepo

```
1 package DAO;
2
3 import java.sql.Connection;
4
5
6 public class OrderDetailRepo implements OrderDetailDAO {
7     private Connection connection;
8
9     final String insert = "INSERT INTO order_detail(id_order_detail, id_order, id_layanan, jumlah, total) VALUES(?, ?, ?, ?, ?)";
10    final String select = "SELECT * FROM order_detail";
11    final String delete = "DELETE FROM order_detail WHERE id_order_detail=?";
12    final String update = "UPDATE order_detail SET id_order=?, id_layanan=?, jumlah=?, total=? WHERE id_order_detail=?";
13
14
15    public OrderDetailRepo() {
16        connection = Database.koneksi();
17    }
18
19    @Override
20    public void save(OrderDetail orderDetail) {
21        PreparedStatement st = null;
22        try {
23            st = connection.prepareStatement(insert);
24            st.setString(1, orderDetail.getIdOrderDetail());
25            st.setString(2, orderDetail.getIdOrder());
26            st.setString(3, orderDetail.getIdLayanan());
27            st.setInt(4, orderDetail.getJumlah());
28            st.setDouble(5, orderDetail.getTotal());
29            st.executeUpdate();
30        } catch (SQLException e) {
31            e.printStackTrace();
32        } finally {
33            try {
34                if (st != null) st.close();
35            } catch (SQLException e) {
36                e.printStackTrace();
37            }
38        }
39    }
40
41    @Override
42    public List<OrderDetail> show() {
43        List<OrderDetail> orderDetails = new ArrayList<>();
44        Statement st = null;
45        ResultSet rs = null;
46        try {
47            st = connection.createStatement();
48            rs = st.executeQuery(select);
49            while (rs.next()) {
50                OrderDetail orderDetail = new OrderDetail();
51                orderDetail.setIdOrderDetail(rs.getString("id_order_detail"));
52                orderDetail.setIdOrder(rs.getString("id_order"));
53                orderDetail.setIdLayanan(rs.getString("id_layanan"));
54                orderDetail.setJumlah(rs.getInt("jumlah"));
55                orderDetail.setTotal(rs.getDouble("total"));
56                orderDetails.add(orderDetail);
57            }
58        } catch (SQLException e) {
59            Logger.getLogger(OrderDetailDAO.class.getName()).log(Level.SEVERE, null, e);
60        } finally {
61            try {
62                if (rs != null) rs.close();
63                if (st != null) st.close();
64            } catch (SQLException e) {
65                e.printStackTrace();
66            }
67        }
68        return orderDetails;
69    }
70
71    @Override
72    public void update(OrderDetail orderDetail) {
73        PreparedStatement st = null;
74        try {
75            st = connection.prepareStatement(update);
76            st.setString(1, orderDetail.getIdOrder());
77            st.setInt(2, orderDetail.getJumlah());
78            st.setDouble(3, orderDetail.getTotal());
79            st.setString(4, orderDetail.getIdOrderDetail());
80            st.executeUpdate();
81        } catch (SQLException e) {
82            e.printStackTrace();
83        } finally {
84            try {
85                if (st != null) st.close();
86            } catch (SQLException e) {
87                e.printStackTrace();
88            }
89        }
90    }
91
92    @Override
93    public void delete(String idOrderDetail) {
94        PreparedStatement st = null;
95        try {
96            st = connection.prepareStatement(delete);
97            st.setString(1, idOrderDetail);
98            st.executeUpdate();
99        } catch (SQLException e) {
100            e.printStackTrace();
101        } finally {
102            try {
103                if (st != null) st.close();
104            } catch (SQLException e) {
105                e.printStackTrace();
106            }
107        }
108    }
109
110    @Override
111    public List<OrderDetail> findAll() {
112        return null;
113    }
114 }
```

3. Membuat Class OrderDetail pada Package model

```
1 package model;
2
3 public class OrderDetail {
4     private String idOrderDetail;
5     private String idOrder;
6     private int jumlah;
7     private double total;
8     private String idLayanan;
9
10    public String getIdLayanan() {
11        return idLayanan;
12    }
13
14    public void setIdLayanan(String idLayanan) {
15        this.idLayanan = idLayanan;
16    }
17
18    public String getIdOrderDetail() {
19        return idOrderDetail;
20    }
21
22    public void setIdOrderDetail(String idOrderDetail) {
23        this.idOrderDetail = idOrderDetail;
24    }
25
26    public String getIdOrder() {
27        return idOrder;
28    }
29
30    public void setIdOrder(String idOrder) {
31        this.idOrder = idOrder;
32    }
33
34    public int getJumlah() {
35        return jumlah;
36    }
37
38    public void setJumlah(int jumlah) {
39        this.jumlah = jumlah;
40    }
41
42    public double getTotal() {
43        return total;
44    }
45
46    public void setTotal(double total) {
47        this.total = total;
48    }
49 }
50
```

4. Membuat Class TableOrderDetail pada Package table

```
1 package table;
2
3 import javax.swing.table.AbstractTableModel;
4
5
6
7 public class TableOrderDetail extends AbstractTableModel {
8     List<OrderDetail> orderDetail;
9     private String[] columnNames = {"ID Order Detail", "ID Order", "ID Layanan", "Jumlah", "Total"};
10
11    public TableOrderDetail(List<OrderDetail> orderDetail) {
12        this.orderDetail = orderDetail;
13    }
14
15    @Override
16    public int getRowCount() {
17        return orderDetail.size();
18    }
19
20    @Override
21    public int getColumnCount() {
22        return columnNames.length;
23    }
24
25    @Override
26    public String getColumnHeader(int column) {
27        return columnNames[column];
28    }
29
30    @Override
31    public Object getValueAt(int rowIndex, int columnIndex) {
32        OrderDetail orderDetail = orderDetail.get(rowIndex);
33        switch (columnIndex) {
34            case 0:
35                return orderDetail.getIdOrderDetail();
36            case 1:
37                return orderDetail.getIdOrder();
38            case 2:
39                return orderDetail.getIdLayanan();
40            case 3:
41                return orderDetail.getJumlah();
42            case 4:
43                return orderDetail.getTotal();
44            default:
45                return null;
46        }
47    }
48 }
49
```

6. Membuat Class OrderDetailFrame di Package ui

Order ID

Pelanggan

Tanggal

Tanggal Pengembalian

Status

Total

Rp. 10.000

Pembayaran

Status Pembayaran

Layanan

HargaKg

Jumlah

Total

Simpan

Update

Hapus

Batal

Simpan

Simpan

5. Membuat Logika pada TextField yang Bervariable textJumlah

```
textJumlah = new JTextField();
textJumlah.addKeyListener(new KeyAdapter() {
    @Override
    public void keyReleased(KeyEvent e) {
        double hargaKg = Double.parseDouble(textHargakg.getText());
        int jumlah = Integer.parseInt(textJumlah.getText());
        double total = hargaKg * jumlah;
        textTotal.setText(String.valueOf(total));
    }
});
textJumlah.setColumns(10);
textJumlah.setBounds(277, 215, 126, 20);
contentPane.add(textJumlah);
```

6. Membuat Fungsi Create pada btnSave

```
JButton btnSave = new JButton("Simpan");
btnSave.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        OrderDetail detail = new OrderDetail();
        detail.setidLayanan(id);
        detail.setidOrder(txtidOrder.getText());
        int jumlah = Integer.parseInt(textJumlah.getText());
        detail.setJumlah(jumlah);
        double total = Double.parseDouble(textTotal.getText());
        detail.setTotal(total);
        orderDetailRepo.save(detail);
        reset();
        loadTableOrderDetail();
    }
});
btnSave.setBounds(277, 252, 89, 23);
contentPane.add(btnSave);
```

7. Membuat Fungsi Update pada btnUpdate

```
JButton btnUpdate = new JButton("Update");
btnUpdate.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        OrderDetail detail = new OrderDetail();
        detail.setIdLayanan(id);
        detail.setIdOrder(txtIdOrder.getText());
        int jumlah = Integer.parseInt(textJumlah.getText());
        detail.setJumlah(jumlah);
        double total = Double.parseDouble(textTotal.getText());
        detail.setTotal(total);
        orderDetailRepo.save(detail);
        reset();
        loadTableOrderDetail();
    }
});
btnUpdate.setBounds(376, 252, 89, 23);
contentPane.add(btnUpdate);
```

8. Membuat Fungsi Delete pada btnDelete

```
JButton btnDelete = new JButton("Hapus");
btnDelete.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        if(id1 != null) {
            orderDetailRepo.delete(id1);
            reset();
            loadTableOrderDetail();
        } else {
            JOptionPane.showMessageDialog(null, "Silahkan Pilih Data yang Akan di Hapus");
        }
    }
});
btnDelete.setBounds(478, 252, 89, 23);
contentPane.add(btnDelete);
```

9. Membuat Fungsi pada tableLayanan

```
tableLayanan = new JTable();
tableLayanan.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        id = tableLayanan.getValueAt(tableLayanan.getSelectedRow(), 0).toString();
        textHargakg.setText(tableLayanan.getValueAt(tableLayanan.getSelectedRow(), 2).toString());
    }
});
tableLayanan.setBounds(276, 45, 323, 180);
contentPane.add(tableLayanan);
```

10. Membuat Fungsi untuk tableOrderDetail

```
tableOrderDetail = new JTable();
tableOrderDetail.addMouseListener(new MouseAdapter() {
    @Override
    public void mouseClicked(MouseEvent e) {
        id = tableLayanan.getValueAt(tableLayanan.getSelectedRow(), 2).toString();
        id1 = tableOrderDetail.getValueAt(tableOrderDetail.getSelectedRow(), 0).toString();
        textJumlah.setText(tableOrderDetail.getValueAt(tableOrderDetail.getSelectedRow(), 3).toString());
        textTotal.setText(tableOrderDetail.getValueAt(tableOrderDetail.getSelectedRow(), 4).toString());
        txtIdOrder.setText(tableOrderDetail.getValueAt(tableOrderDetail.getSelectedRow(), 1).toString());
    }
});
tableOrderDetail.setBounds(277, 304, 323, 138);
contentPane.add(tableOrderDetail);
```

11. Method tambahan untuk Load Table dari Database

```
public void reset() {
    txtIdOrder.setText("");
    textJumlah.setText("");
    textTotal.setText("");
}

ServiceRepo srvc = new ServiceRepo();
List<Service> ls;
public String id;

OrderDetailRepo orderDetailRepo = new OrderDetailRepo();
List<OrderDetail> ls1;
public String id1;
private JTable tableOrderDetail;

public void loadTableOrderDetail() {
    ls1 = orderDetailRepo.show();
    TableOrderDetail tu = new TableOrderDetail(ls1);
    tableOrderDetail.setModel(tu);
    tableOrderDetail.getTableHeader().setVisible(true);
}

public void loadTable() {
    ls = srvc.show();
    TableService tu = new TableService(ls);
    tableLayanan.setModel(tu);
    tableLayanan.getTableHeader().setVisible(true);
}
```

12.Hasilnya Seperti ini :

Order ID

Pelanggan

Pilih

Tanggal

Tanggal Pengembalian

Status

Proses

Total

Rp. 10.000

Pembayaran

Cash

Status Pembayaran

Lunas

Simpan

Simpan

Layanan

ID	Jenis	Harga	Status
1	cuci	10000.0	menunggu
2	cuci	20000.0	menunggu

HargaKg

Jumlah

Total

Simpan

Update

Hapus

Batal

ID Order D...	ID Order	ID Layanan	Jumlah	Total
2	trx-001		2	20000.0
3			2	40000.0
4			2	20000.0
5	trx-002		5	100000.0
6	trx-003		2	20000.0
7	trx-005		2	20000.0
8	trx-003		3	30000.0
9	trx66	1	2	20000.0
10	trx666	2	2	40000.0
11	trx00001	2	2	40000.0